MATH 6641 Final Project

A Continuous Finite Element Method for the Fokker-Planck Equation

Brendan Shrader

Table of Contents

Section 1: Introduction . . . 2
Section 2: The General Finite Element Method . . . 2
2.1. Solvability . . . 3
2.1.1. Implicit Euler Time Discretization . . . 3
2.1.2. Crank-Nicholson Time Discretization . . . 4
2.2. The Linear System . . . 5
2.2.1. Computational Complexity of Solving the Linear System . . . 6
2.2.2. Pseudocode for a 2D Rectangular Triangulation . . . 7
Section 3: The Finite Element Method in 1D . . . 9
3.1. Linear Test Functions and Crank-Nicholson Euler Time Discretization . . . 9
3.1.1. Linear Test Functions and Implicit Euler Time Discretization . . . 11
3.2. Numerical Simulations of the 1D Implicit Euler FEM . . . 12
3.2.1. Ornstein-Uhlenbeck Process Simulations . . . 13
3.2.2. Cox-Ingersoll-Ross Process Simulations . . . 13
Section 4: Discussion . . . 15

1. Introduction

The Fokker-Planck equation describes the evolution of a probability density function for a diffusion process, or in our case, a stochastic differential equation (SDE). Let $\mathbf{x}(t) \in \Omega \subseteq \mathbb{R}^n$ denote the state variables, and let

$$\boldsymbol{\mu}(\boldsymbol{x}, t) : \Omega \times [0, T] \to \mathbb{R}^n \quad \text{and} \quad \boldsymbol{\sigma}(\boldsymbol{x}, t) : \Omega \times [0, T] \to \mathbb{R}^{n \times m}.$$

We call $\boldsymbol{\mu}(\boldsymbol{x}, t)$ the "drift" term and $\boldsymbol{\sigma}(\boldsymbol{x}, t)$ the "noise" or "diffusion" term. We consider a general SDE of the form

$$d\boldsymbol{x}(t) = \boldsymbol{\mu}(\boldsymbol{x}, t)\, dt + \boldsymbol{\sigma}(\boldsymbol{x}, t)\, d\boldsymbol{w}_t, \tag{1}$$

where $d\boldsymbol{w}_t$ is the Itô differential of an m-dimensional Wiener process (or Brownian motion). Note that $\boldsymbol{\sigma}(\boldsymbol{x}, t)$ may not be square if the number of sources of noise is less than or greater than the number of state variables. For example, if the SDE describes a chemical reaction network, then there are generally more reactions terms than the number of variables, and so $m \geq n$. Intuitively, this stochastic differential equation is guided by the drift term $\boldsymbol{\mu}(\boldsymbol{x}, t)$, similar to an ODE is, but with additional random noise described by $\boldsymbol{\sigma}(\boldsymbol{x}, t)$. Both the drift and noise may depend on the current state, $\boldsymbol{x}$, and the current time, t. Given that $d\boldsymbol{w}_t$ is a random variable, a solution $\boldsymbol{x}(t)$ to this SDE is also a random variable, and hence has a probability distribution. This probability distribution, $p(\boldsymbol{x}, t)$, is described by the **Fokker-Planck equation**,

$$\frac{\partial p(\boldsymbol{x}, t)}{\partial t} = -\sum_{i=1}^{n} \frac{\partial}{\partial x_i}\big[\boldsymbol{\mu}(\boldsymbol{x}, t) p(\boldsymbol{x}, t)\big] + \sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\partial^2}{\partial x_i \partial x_j}\big[\boldsymbol{D}_{ij}(\boldsymbol{x}, t) p(\boldsymbol{x}, t)\big] \quad \text{where} \quad \boldsymbol{D} = \frac{1}{2}\boldsymbol{\sigma}\boldsymbol{\sigma}^T. \tag{2}$$

In this project, I will derive and study the solvability of a continuous Galerkin method for the general Fokker-Planck equation. Then, I will derive and explain how to solve the resulting linear system. Finally, I will explicitly compute the finite element method in the 1D case using both implicit Euler and Crank-Nichoslon time discretizations and will simulate two standard, analytically solvable Fokker-Planck equations.

2. The General Finite Element Method

For notation, let $\nabla \cdot \boldsymbol{D}$ be the column vector whose i^{th} entry is the divergence of the ith row of $\boldsymbol{D}$. Then we can write the Fokker-Planck equation more succinctly as

$$p_t = -\nabla \cdot \boldsymbol{J}[p] \quad \text{where} \quad \boldsymbol{J}[p] = \boldsymbol{\mu} p - \boldsymbol{D}\nabla p - (\nabla \cdot \boldsymbol{D}) p \tag{3}$$

The domain for $\boldsymbol{x}$ is $\Omega \subset \mathbb{R}^n$, and the domain for t is $[0, T]$. At the boundary $\partial\Omega$, we will use a zero-flux boundary condition

$$\boldsymbol{J}[p] \cdot \vec{n}\,\Big|_{\partial\Omega} = 0,$$

and so we will not include any boundary conditions in the finite element space. Instead, the boundary condition will be used in the derivation of the finite element method. Hence, the (continuous) finite element space is

$$\mathbf{V}_h = \left\{\varphi \in C(\Omega) : \varphi|_\tau \in \mathbb{P}_r(\tau) \text{ for each } \tau \in \mathcal{T}_h\right\},$$

where $\mathcal{T}_h$ is a sufficiently nice triangulation of Ω and $\mathbb{P}_r(\tau)$ is the set of all degree r polynomials on $\tau \in \mathcal{T}_h$. Multiplying (3) by $\varphi \in \mathbf{V}_h$, integrating by parts, and using the zero-flux boundary condition,

$$\int_\Omega p_t \varphi = -\int_\Omega (\nabla \cdot \boldsymbol{J}[p])\varphi = -\boldsymbol{J}[p]\varphi\Big|_{\partial\Omega} + \int_\Omega \boldsymbol{J}[p] \cdot \nabla\varphi = \int_\Omega \boldsymbol{J}[p] \cdot \nabla\varphi \tag{4}$$

Using bilinear form notation, this is equivalent to

$$(p_t, \varphi) = (\boldsymbol{J}[p], \nabla\varphi).$$

Note that $\boldsymbol{J}[p]$ only has a first derivative of p (the gradient operator), and so we can replace p by $P(\cdot, t) \in \mathbf{V}_h$ and obtain the variational form of the finite element method.

Variational Form of the Finite Element Method

Find $P(\cdot, t) \in \mathbf{V}_h$ such that

$$(P_t, \varphi) = (\boldsymbol{J}[P], \nabla\varphi) \quad \text{for all } \varphi \in \mathbf{V}_h, \tag{5}$$

where

$$\boldsymbol{J}[P] = \boldsymbol{\mu}P - \boldsymbol{D}\nabla P - (\nabla \cdot \boldsymbol{D})P \quad \text{and} \quad \boldsymbol{D} = \frac{1}{2}\boldsymbol{\sigma}\boldsymbol{\sigma}^T.$$

2.1: Solvability

2.1.1: Implicit Euler Time Discretization

Consider the implicit Euler time discretization

$$\left(\frac{P^{n+1} - P^n}{\Delta t}, \varphi\right) = (\boldsymbol{J}^{n+1}[P^{n+1}], \nabla\varphi) \quad \text{for all } \varphi \in \mathbf{V}_h. \tag{6}$$

This method has a unique solution P^{n+1} if and only if the homogeneous system has a unique solution $P^{n+1} \equiv 0$. The corresponding homogeneous system is

$$(P^{n+1}, \varphi) - \Delta t(\boldsymbol{J}^{n+1}[P^{n+1}], \nabla\varphi) = 0. \tag{7}$$

Intuitively, the inner product (P^{n+1}, φ) is positive definite, so this method should (certainly) have a unique solution when Δt is very small. The following theorem gives a sufficient condition for the solvability of (7).

Theorem 2.1. *The homogeneous system (7) has a unique solution if $\Delta t < h/MC$ (the CFL condition), where h is the mesh size,*

$$M = \sup_{(\boldsymbol{x}, t) \in \Omega \times [0, T]} \|\boldsymbol{\mu}(\boldsymbol{x}, t) - \nabla \cdot \boldsymbol{D}(\boldsymbol{x}, t)\|$$

and C is the constant such that $\|\nabla\varphi\|_{L^2} \leq Ch^{-1}\|\varphi\|_{L^2}$.

Proof. Since the homogeneous equation (7) must hold for all $\varphi \in \mathbf{V}_h$, take $\varphi \equiv P^{n+1}$. Then (7) becomes

$$\|P^{n+1}\|_{L^2}^2 - \Delta t(\boldsymbol{J}^{n+1}[P^{n+1}], \nabla P^{n+1}) = 0. \tag{8}$$

We will show that if $\Delta t < h/MC$, then the left-hand side of (8) is positive whenever $P^{n+1} \neq 0$. To do so, write

$$\begin{aligned}(\boldsymbol{J}^{n+1}[P^{n+1}], \nabla P^{n+1}) &= \int_\Omega \boldsymbol{\mu}^{n+1} P^{n+1} \cdot \nabla P^{n+1} - \int_\Omega (\boldsymbol{D}^{n+1} \nabla P^{n+1}) \cdot \nabla P^{n+1} - \int_\Omega (\nabla \cdot \boldsymbol{D}^{n+1}) P^{n+1} \cdot \nabla P^{n+1} \\ &= \int_\Omega (\boldsymbol{\mu}^{n+1} - \nabla \cdot \boldsymbol{D}^{n+1}) P^{n+1} \cdot \nabla P^{n+1} - \int_\Omega (\boldsymbol{D}^{n+1} \nabla P^{n+1}) \cdot \nabla P^{n+1}.\end{aligned}$$

For the second integral, note that

$$(\boldsymbol{D}^{n+1} \nabla P^{n+1}) \cdot \nabla P^{n+1} = (\nabla P^{n+1})^T \boldsymbol{\sigma} \boldsymbol{\sigma}^T \nabla P^{n+1} = \left[(\nabla P^{n+1})^T \boldsymbol{\sigma}\right]\left[(\nabla P^{n+1})^T \boldsymbol{\sigma}\right]^T,$$

which is a row vector times its transpose, and hence is nonnegative. Therefore, by the Cauchy-Schwarz inequality and the fact that $\|fg\|_{L^2} \leq \|f\|_{L^\infty} \|g\|_{L^2}$

$$\begin{aligned}(\boldsymbol{J}^{n+1}[P^{n+1}], \nabla P^{n+1}) &\leq \int_\Omega (\boldsymbol{\mu}^{n+1} - \nabla \cdot \boldsymbol{D}^{n+1}) P^{n+1} \cdot \nabla P^{n+1} \\ &= \left\| (\boldsymbol{\mu}^{n+1} - \nabla \cdot \boldsymbol{D}^{n+1}) P^{n+1} \cdot \nabla P^{n+1} \right\|_{L^2} \\ &\leq \left\| (\boldsymbol{\mu}^{n+1} - \nabla \cdot \boldsymbol{D}^{n+1}) P^{n+1} \right\|_{L^2} \left\| \nabla P^{n+1} \right\|_{L^2} \\ &\leq \left\| (\boldsymbol{\mu}^{n+1} - \nabla \cdot \boldsymbol{D}^{n+1}) \right\|_{L^\infty} \left\| P^{n+1} \right\|_{L^2} \left\| \nabla P^{n+1} \right\|_{L^2}.\end{aligned}$$

We know that

$$\left\| (\boldsymbol{\mu}^{n+1} - \nabla \cdot \boldsymbol{D}^{n+1}) \right\|_{L^\infty} = \sup_{(\boldsymbol{x},t) \in \Omega \times [0,T]} \left\| (\boldsymbol{\mu}^{n+1}(\boldsymbol{x},t) - \nabla \cdot \boldsymbol{D}^{n+1}(\boldsymbol{x},t)) \right\| \leq M$$

and $\|\nabla P^{n+1}\|_{L^2} \leq Ch^{-1} \|P^{n+1}\|_{L^2}$, and so

$$(\boldsymbol{J}^{n+1}[P^{n+1}], \nabla P^{n+1}) \leq MCh^{-1} \left\| P^{n+1} \right\|_{L^2}^2.$$

Therefore, if $\Delta t < h/MC$, then

$$\Delta t (\boldsymbol{J}^{n+1}[P^{n+1}], \nabla P^{n+1}) < \left\| P^{n+1} \right\|_{L^2}^2,$$

and so (8) holds if and only if $P^{n+1} \equiv 0$. Therefore, the implicit Euler method has a unique solution when $t < MCh^{-1}$. □

Remark 2.1. (a) The CFL condition $\Delta t < h/MC$ is effectively the CFL condition, as it depends on the mesh size h and the effective maximum characteristic speed M. The constant C depends only on the structuring of the mesh, not on the domain or mesh size.

(b) The CFL condition is only a sufficient condition for the solvability of (6). By numerical experiments, the method does not actually require the CFL condition to hold. This means that my method of proof is not the best way to prove solvability. Instead, it guarantees that coding the numerical method is worthwhile because I know that I can obtain a unique solution, although numerical experiments later reveal that the CFL condition is not necessary.

2.1.2: Crank-Nicholson Time Discretization

The Crank-Nicholson time discretization is

$$\left(\frac{P^{n+1}-P^n}{\Delta t},\varphi\right)=\frac{1}{2}(\boldsymbol{J}^{n+1}[P^{n+1}]+\boldsymbol{J}^n[P^n],\nabla\varphi)\quad\text{for all }\varphi\in\mathbf{V}_h, \tag{9}$$

and the corresponding homogeneous system is

$$(P^{n+1},\varphi)=\frac{\Delta t}{2}(\boldsymbol{J}^{n+1}[P^{n+1}],\nabla\varphi),$$

which is the exact same as (7) with a factor of 1/2. Hence, the above proof also guarantees a unique solution for this Crank-Nicholson method for $\Delta t < h/2MC$, although once again, this CFL condition is not necessary.

2.2: The Linear System

Consider the Crank-Nicholson time discretization (9). It can be flattened (with implicit terms on the left hand side and explicit terms on the right hand side) as

$$(P^{n+1},\varphi)-\frac{\Delta t}{2}(\boldsymbol{J}^{n+1}[P^{n+1}],\nabla\varphi)=(P^n,\varphi)+\frac{\Delta t}{2}(\boldsymbol{J}^n[P^n],\nabla\varphi)\quad\text{for all }\varphi\in\mathbf{V}_h \tag{10}$$

The finite element space $\mathbf{V}_h$ has a finite set of basis functions $\beta=\{\varphi_i : 1\le i\le N_\beta\}$. To obtain the linear system, we write P^n as a linear combination of these basis functions for each n,

$$P^n=\sum_{i=1}^{N_\beta}c_i^n\varphi_i.$$

Also, if (10) holds for all $\varphi_i\in\beta$, then it holds for all $\varphi\in\mathbf{V}_h$, so we only need to consider (10) for $\varphi_i\in\beta$. Since $\boldsymbol{J}[P]$ is linear with respect to P, so the flux inner products then become

$$(\boldsymbol{J}^{n+1}[P^{n+1}],\nabla\varphi_j)=c_i^{n+1}(\boldsymbol{J}^{n+1}[\varphi_i],\nabla\varphi_j)\quad\text{and}\quad(\boldsymbol{J}^n[P^n],\nabla\varphi_j)=c_i^n(\boldsymbol{J}^n[\varphi_i],\nabla\varphi_j).$$

Now, the Crank-Nicholson scheme is to find $\{c_i^{n+1}\}$ (given $\{c_i^n\}$) such that

$$\sum_{i=1}^{N_\beta}\left[c_i^{n+1}(\varphi_i,\varphi_j)-\frac{\Delta t}{2}c_i^{n+1}(\boldsymbol{J}^{n+1}[\varphi_i],\nabla\varphi_j)\right]=\sum_{i=1}^{N_\beta}\left[c_i^n(\varphi_i,\varphi_j)+\frac{\Delta t}{2}c_i^n(\boldsymbol{J}^n[\varphi_i],\nabla\varphi_j)\right].$$

Now, what do the inner products $(\boldsymbol{J}^n[\varphi_i],\nabla\varphi_j)$ look like? We can write it as

$$(\boldsymbol{J}^n[\varphi_i],\nabla\varphi_j)=(\boldsymbol{\mu}^n\varphi_i,\nabla\varphi_j)-(\boldsymbol{D}^n\nabla\varphi_i,\nabla\varphi_j)-\left((\nabla\cdot\boldsymbol{D}^n)\varphi_i,\nabla\varphi_j\right)$$

Now, we are ready to define the matrices that constitute the linear system:

$$M=(m_{ij})\quad\text{where}\quad m_{ij}=(\varphi_j,\varphi_i)=\int_\Omega\varphi_j\varphi_i, \tag{11a}$$

$$C^n=(c_{ij}^n)\quad\text{where}\quad c_{ij}^n=(\boldsymbol{\mu}^n\varphi_j,\nabla\varphi_i)=\int_\Omega\boldsymbol{\mu}^n\varphi_j\cdot\nabla\varphi_i, \tag{11b}$$

$$K^n=(k_{ij}^n)\quad\text{where}\quad k_{ij}^n=(\boldsymbol{D}^n\nabla\varphi_j,\nabla\varphi_i)=\int_\Omega(\nabla\varphi_i)^T\boldsymbol{D}^n(\nabla\varphi_j), \tag{11c}$$

$$B^n=(b_{ij}^n)\quad\text{where}\quad b_{ij}^n=\left((\nabla\cdot\boldsymbol{D}^n)\varphi_j,\nabla\varphi_i\right)=\int_\Omega(\nabla\cdot\boldsymbol{D}^n)\varphi_j\cdot\nabla\varphi_i. \tag{11d}$$

With this, the linear system for the Crank-Nicholson method is

$$\left(M - \frac{\Delta t}{2} A^{n+1}\right)\vec{c}^{\,n+1} = \left(M + \frac{\Delta t}{2} A^{n}\right)\vec{c}^{\,n} \quad \text{where} \quad A^n = C^n - K^n - B^n. \tag{12}$$

The M matrix only needs to be computed at the beginning of the simulation, but depending on how $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ depend on $\boldsymbol{x}$ and t, these C^n, K^n, and B^n matrices can be easy or difficult to compute. There are four cases:

(a) In the worst case, if $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ depend on $\boldsymbol{x}$ and t, then C^n, K^n, and B^n must be recomputed at each time step.

(b) If $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ only depend on $\boldsymbol{x}$, then C^n, K^n, and B^n are constant with respect to n and hence only need to be computed at the beginning of the simulation.

(c) If $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ only depend on t, then c_{ij} and b_{ij} can be computed more simply at each time step as

$$c_{ij}^n = \boldsymbol{\mu}^n \cdot \int_\Omega \varphi_j(\nabla\varphi_i) \quad \text{and} \quad b_{ij}^n = (\nabla\cdot\boldsymbol{D}^n)\cdot\int_\Omega \varphi_j(\nabla\varphi_i),$$

where these two integrals are only computed once upon initialization, but K^n must be computed using integration at each time step.

(d) If $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are constant, then all matrices must only be computed once at the beginning of the simulation, and their entries are simply

$$c_{ij} = \boldsymbol{\mu}^0 m_{ij}, \quad k_{ij} = \int_\Omega (\nabla\varphi_j)^T \boldsymbol{D}^0 (\nabla\varphi_i), \quad \text{and} \quad b_{ij} = 0.$$

Also, note that the entries can be defined using integrals over all of Ω, but integrating over the whole domain is essentially never necessary (unless the mesh is very bad), and in the program we can store the vertices and triangles of the mesh in such a way that the entries need only be computed over 2 triangles (if $i \neq j$) and 6 triangles (if $i = j$) for the mesh in Figure 1.

2.2.1: Computational Complexity of Solving the Linear System

As an example, consider a rectangular domain $\Omega = [0, L_1] \times [0, L_2] \subset \mathbb{R}^2$ with the structured triangular mesh in Figure 1. Each nodal basis function (on the interior) is nonzero on 6 neighboring triangles, and so each nodal basis function shares a nonzero region of the domain with at most 6 neighboring nodal basis functions. This means that each row (and column) of the matrices M, C^n, K^n, and B^n will have at most 7 entries, and so all matrices will be sparse and will not have any diagonal structuring. Upon starting the simulation, one can compute which entries of the matrices will be nonzero, and this data can be stored in a

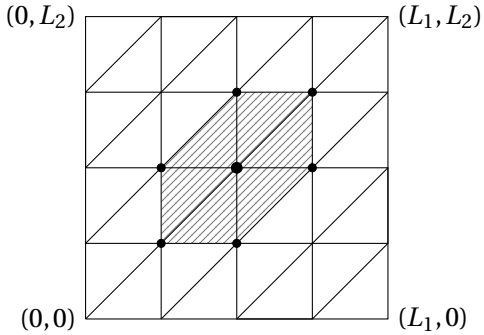


Figure 1: 2D FEM Triangulation

data structure (for each nodal basis function, we can store the neighboring basis functions and the triangles on which both are defined). Then, even in the worst case that $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ depend on $\boldsymbol{x}$ and t, the computational complexity of computing C^n, K^n, and B^n at each timestep will be $O(N_\beta)$, where N_β is the number of nodal basis functions. Moreover, sparse matrix solvers can achieve close to $O(N_\beta)$ time complexity, so each timestep should take $O(N_\beta)$ computations on average. Even if the triangulation is unstructured, as long as there is a minimum angle θ_0 among all triangles, then each nodal basis function will share a triangle with only finitely many other nodal basis functions, and so the time complexity should still be $O(N_\beta)$. If $\theta_0 \ll 1$, then it is possible for the time complexity to perform slower than $O(N_\beta)$, as many nodal basis functions will share many triangles with many other nodal basis functions, making the matrices less sparse.

2.2.2: Pseudocode for a 2D Rectangular Triangulation

Given a set of dimensions = $\big[[0, L_1], [0, L_2]\big]$ and a step size h, we first generate a mesh in the form of Figure 1. This mesh contains several important pieces of information including

- The number and location of all vertices.
- The number and coordinates of all triangles.
- A 2D list storing the set of triangles on which a pair of nodal basis functions i and j are both nonzero.

While this takes up a lot of memory, this allows for efficient computation of the M, C, K, and B matrices. The general finite element method in Algorithm 1 is quite simple: generate the mesh, initialize the solution, and iteratively solve the Crank-Nicholson linear system. Algorithm 2 to solve the Crank-Nicholson linear system is also quite simple: generate C^{n+1}, K^{n+1}, and B^{n+1} and solve the linear system (12). We use scipy's implementation of sparse matrices to reduce the space and time complexity of this linear system. Algorithm 3 gives more details on how C^{n+1}, K^{n+1}, and B^{n+1} are computed more efficiently using the 2D list of triangles shared by nodal basis functions stored in the mesh data structure.

Algorithm 1 General Finite Element Method Algorithm

Require: dimensions, h ▷ Require the x and y intervals and max step size h
  mesh ← generateMesh(dimensions, h) ▷ Generate a mesh as in Figure 1
  P, T ← initializeSolution(mesh, tmax, dt, p_0) ▷ Initialize solution array with the initial condition
  M ← generateM(mesh) ▷ M matrix only needs to be generated once
  for n in range(T) **do**
    $P[n+1]$, matrices ← CN_FEMstep($P[n]$, $\boldsymbol{\mu}$, $\boldsymbol{D}$, n, dt, mesh, M, matrices)
  end for
  return P, mesh

The most complex functions in this whole algorithm are the mesh-generating function and the ones which calculate the entries of C^{n+1}, K^{n+1}, and B^{n+1}. For the mesh-generating function, we can do as follows:

(a) Determine the gridpoints of the mesh, taking into account the maximum step size h and dimensions of the domain.

Algorithm 2 CN_FEMstep

Require: $P[n]$, $\boldsymbol{\mu}$, $\boldsymbol{D}$, n, dt, mesh, M, old_matrices
  new_matrices ← generateNewMatrices($\boldsymbol{\mu}$, $\boldsymbol{D}$, $n+1$, mesh) ▷ new_matrices = $[C_{\text{new}}, -K_{\text{new}}, -B_{\text{new}}]$
  left_matrix ← $M - \Delta t *$ np.sum(new_matrices)/2 ▷ Set up LHS of (12)
  right_matrix ← $M + \Delta t *$ np.sum(old_matrices)/2 ▷ Set up RHS of (12)
  P_{new} ← scipy.linalg.sparse.spsolve(left_matrix, right_matrix @ $P[n]$) ▷ Solve linear system
  return P_{new}, new_matrices

Algorithm 3 generateNewMatrices

Require: $\boldsymbol{\mu}$, $\boldsymbol{D}$, $n+1$, mesh
  C, K, B ← scipy.sparse.lilmatrix((mesh.N, mesh.N)) ▷ Allocate space for sparse matrices
  for i in range(N) **do**
    nonzero_cols ← mesh.nonzero_cols[i]
    for j in nonzero_cols **do**
      common_triangles = get_common_triangles(i, j, mesh)
      $C[i,j]$ ← calculate_C_entry($\boldsymbol{\mu}$, $n+1$, i, j, common_triangles)
      $K[i,j]$ ← calculate_K_entry($\boldsymbol{D}$, $n+1$, i, j, common_triangles)
      $B[i,j]$ ← calculate_B_entry($\boldsymbol{D}$, $n+1$, i, j, common_triangles)
    end for
  end for
  return C, K, B

(b) Based on the gridpoints, generate a list of triangles and their vertices.

(c) For each gridpoint, store the triangles for which that gridpoint is a vertex.

(d) For each pair of gridpoints (i, j), store both (1) the indices of the triangles which have both gridpoints as a vertex (i.e., the triangles for which the nodal basis functions φ_i and φ_j are nonzero).

(e) Lastly, for each gridpoint i, find the set of all other gridpoints j that share a triangle using the previously computed data.

This is not the most efficient way to generate all mesh information, but it is the most direct. To calculate the entries of the C, K, and B matrices, we are given the required function (either $\boldsymbol{\mu}$ or $\boldsymbol{D}$), the time level, the vertices (i, j), and the set of all triangles which share these vertices. If $\mathbf{V}_h$ consists of piecewise linear functions, we can define φ_i and φ_j using barycentric coordinates as we did in class, and we can use a suitable numerical method to compute the necessary integrals (e.g. Gaussian quadrature). Furthermore, if $\mathbf{V}_h$ consists of piecewise linear functions, then the gradients $\nabla\varphi_i$ will be constant over each triangle, and if the mesh is generated in such a way that it is uniform and translationally symmetric, then the entries k_{ij} will be easy to compute. If $\mathbf{V}_h$ consists of higher degree polynomials or if the mesh is not uniform, then $\nabla\varphi_i$ will need to be computed over each triangle numerically, but this data can also be stored within the mesh. Furthermore, if $\mathbf{V}_h$ consists of higher degree polynomials, then P^n in each triangle will have six coefficients, and so we will need to define nodal basis functions at additional points in the domain (for example, we can define nodal basis functions at each vertex and at each midpoint of the triangles).

3. The Finite Element Method in 1D

3.1: Linear Test Functions and Crank-Nicholson Euler Time Discretization

We consider the 1D case because the linear system (12) can be derived explicitly without relying on numerical calculations of the mesh and matrices. In 1D, our domain will be an interval $\Omega = [x_0, x_N]$ consisting of uniform subintervals $I_n = [x_{n-1}, x_n]$ for $1 \le n \le N$ with mesh size $h = x_n - x_{n-1}$. Also, we use non-bold letters for μ and D because they are scalar-valued in the 1D case. Recall that we will use zero-flux boundary conditions at the endpoints, so our linear test function space will not enforce boundary conditions and is thus

$$\mathbf{V}_h = \{\varphi \in \mathcal{C}(\Omega) : \varphi \text{ is linear in each } I_n\}.$$

Since we already know the linear system we must solve, equation (12), we only need to calculate the entries of the matrices. The nodal basis functions for $\mathbf{V}_h$ on this mesh are

$$\varphi_i(x) = \begin{cases} \frac{1}{h}(x - x_{i-1}) & \text{if } x \in [x_{i-1}, x_i] \cap \Omega, \\ -\frac{1}{h}(x - x_{i+1}) & \text{if } x \in [x_i, x_{i+1}] \cap \Omega, \\ 0 & \text{otherwise,} \end{cases}$$

where $0 \le i \le N$. For the entries of M, we compute the integral of φ_i^2 over Ω,

$$\int_\Omega \varphi_i^2 = \int_{x_{i-1}}^{x_{i+1}} \varphi_i^2 = \int_{x_{i-1}}^{x_i} \frac{1}{h^2}(x - x_{i-1})^2 \, dx - \int_{x_i}^{x_{i+1}} \frac{1}{h^2}(x - x_{i+1})^2 \, dx = \frac{2h}{3}, \tag{13}$$

and the integral of $\varphi_i \varphi_{i+1}$ over Ω,

$$\int_\Omega \varphi_i \varphi_{i+1} = \int_{x_i}^{x_{i+1}} \varphi_i \varphi_{i+1} \, dx = -\frac{1}{h^2} \int_{x_i}^{x_{i+1}} (x - x_{i+1})(x - x_i) \, dx = \frac{1}{6h}\left[x_{i+1}^2 - 2x_i x_{i+1} + x_i^2\right] = \frac{h}{6}.$$

At the boundaries of Ω, the integral of adjacent nodal basis functions, φ_i and φ_{i+1}, remains unchanged because it is only over one subinterval I_n, but the integral of φ_0^2 and φ_N^2 will be half of what was calculated in (13) because they will be integrated over only $[x_0, x_1]$ or $[x_{N-1}, x_N]$, respectively. This means that the entries of M are given by

$$m_{ii} = \begin{cases} \frac{2h}{3} & \text{if } 0 < i < N, \\ \frac{h}{3} & \text{if } i = 0, N. \end{cases} \quad \text{and} \quad m_{i,i-1} = m_{i-1,i} = \frac{h}{6} \quad \text{for } 1 \le i \le N.$$

As in equations (11), the entries of C^n are given by

$$c_{ij}^n = \int_\Omega \mu^n \varphi_j (\partial_x \varphi_i)$$

To compute this integral, interpolate $\mu(x, t_n)$ by a piecewise linear function $\overline{\mu}^n(x) \in \mathbf{V}_h$ (i.e., project $\mu(x, t_n)$ into $\mathbf{V}_h$), and write it as

$$\mu(x, t_n) = \overline{\mu}^n(x) + O(h^2) = \sum_{i=0}^{N} m_i^n \varphi_i(x) + O(h^2).$$

Since the approximation error between $\mu(x,t_n)$ and $\overline{\mu}^n(x)$ is $O(h^2)$, the approximation error between the integral of $\mu(x,t_n)$ and $\overline{\mu}^n(x)$ is $O(h^3)$, and so this introduces negligible error. If $i=j$, then

$$\begin{aligned} c_{ii}^n &= \int_\Omega \overline{\mu}^n \varphi_i(\partial_x \varphi_i) \\ &= \frac{1}{h}\int_{x_{i-1}}^{x_i} (m_{i-1}^n \varphi_{i-1} + m_i^n \varphi_i)\varphi_i - \frac{1}{h}\int_{x_i}^{x_{i+1}} (m_i^n \varphi_i + m_{i+1}^n \varphi_{i+1})\varphi_i \\ &= \frac{m_{i-1}^n}{h}\int_{x_{i-1}}^{x_i} \varphi_{i-1}\varphi_i + \frac{m_i^n}{h}\int_{x_{i-1}}^{x_i} \varphi_i^2 - \frac{m_i^n}{h}\int_{x_i}^{x_{i+1}} \varphi_i^2 - \frac{m_{i+1}^n}{h}\int_{x_i}^{x_{i+1}} \varphi_i\varphi_{i+1}. \end{aligned}$$

On the interior, the middle integrals cancel out, and so

$$c_{ii}^n = \frac{m_{i-1}^n}{h}\int_{x_{i-1}}^{x_i} \varphi_{i-1}\varphi_i - \frac{m_{i+1}^n}{h}\int_{x_i}^{x_{i+1}} \varphi_i\varphi_{i+1} = \frac{m_{i-1}^n}{6} - \frac{m_{i+1}^n}{6} \quad \text{for } 0<i<N.$$

On the other hand, at the left boundary, the integrals over $[x_{i-1},x_i]$ disappear; and at the right boundary, the integrals over $[x_i,x_{i+1}]$ disappear, and so

$$\begin{aligned} c_{0,0}^n &= -\frac{m_0^n}{h}\int_{x_0}^{x_1} \varphi_0^2 - \frac{m_1^n}{h}\int_{x_0}^{x_1} \varphi_0\varphi_1 = -\frac{m_0^n}{3} - \frac{m_1^n}{6} \\ c_{N,N}^n &= \frac{m_{N-1}^n}{h}\int_{x_{N-1}}^{x_N} \varphi_{N-1}\varphi_N + \frac{m_N^n}{h}\int_{x_{N-1}}^{x_N} \varphi_N^2 = \frac{m_{N-1}^n}{6} + \frac{m_N^n}{3} \end{aligned}$$

Now, we need the off-diagonals. If $j=i-1$, then

$$\begin{aligned} c_{i,i-1}^n &= \int_\Omega \overline{\mu}^n \varphi_{i-1}(\partial_x\varphi_i) = \frac{1}{h}\int_{x_{i-1}}^{x_i} (m_{i-1}^n\varphi_{i-1} + m_i^n\varphi_i)\varphi_{i-1} = \frac{m_{i-1}^n}{3} + \frac{m_i^n}{6} \quad \text{for } 1\le i\le N, \\ c_{i,i+1}^n &= \int_\Omega \overline{\mu}^n \varphi_{i+1}(\partial_x\varphi_i) = -\frac{1}{h}\int_{x_i}^{x_{i+1}} (m_i^n\varphi_i + m_{i+1}^n\varphi_{i+1})\varphi_{i+1} = -\frac{m_i^n}{6} - \frac{m_{i+1}^n}{3} \quad \text{for } 0\le i\le N-1. \end{aligned}$$

Now, we have all entries of the matrix C^n. Now, we must determine the contribution of the diffusion via the matrices K^n and B^n. To derive the general finite element method, we rewrote the Fokker-Planck equation in divergence form using the flux

$$\boldsymbol{J}[p] = \boldsymbol{\mu} p - \boldsymbol{D}\nabla p - (\nabla\cdot\boldsymbol{D})p.$$

These three terms gave us three matrices, C, K, and B, respectively. This was the neatest way to rewrite the sum of the diffusion terms in (2). However, in 1D, it can be written more simply as

$$\boldsymbol{J}[p] = \mu p - \partial_x(Dp).$$

This means that in 1D, it is easier to combine the K and B matrices into one diffusion term, as when we integrate $\partial_x(Dp)$, we can simply use the fundamental theorem of calculus. More precisely, if we add the entries of K^n and B^n from (11), we obtain

$$\begin{aligned} k_{ij}^n + b_{ij}^n &= \int_\Omega D^n(\partial_x\varphi_j)(\partial_x\varphi_i) + (\partial_x D^n)\varphi_j(\partial_x\varphi_i) \\ &= \int_\Omega \left[D^n(\partial_x\varphi_j) + \varphi_j(\partial_x D^n)\right](\partial_x\varphi_i) \\ &= \int_\Omega \partial_x(D^n\varphi_j)(\partial_x\varphi_i) \end{aligned}$$

Define the new matrix S^n by

$$s_{ij}^n \equiv k_{ij}^n + b_{ij}^n = \int_\Omega \partial_x(D^n\varphi_j)(\partial_x\varphi_i).$$

Then the entries can be computed as

$$s_{ij}^n = \int_{x_{i-1}}^{x_{i+1}} \partial_x(D^n\varphi_j)(\partial_x\varphi_i) = \frac{1}{h}\int_{x_{i-1}}^{x_i} \partial_x(D^n\varphi_j) - \frac{1}{h}\int_{x_i}^{x_{i+1}} \partial_x(D^n\varphi_j) \tag{14}$$

On the interior ($0 < i < N$), both integrals are nonzero, and so

$$s_{ij}^n = \frac{1}{h}\left[-(D^n\varphi_j)|_{x_{i-1}} + 2(D^n\varphi_j)|_{x_i} - (D^n\varphi_j)|_{x_{i+1}}\right]$$

(notice how this looks like a central difference). Since $\varphi_j|_{x_k} = 1$ if and only if $x_k = x_j$ and zero otherwise, we have

$$s_{ii}^n = \frac{2D_i^n}{h}, \qquad s_{i,i-1} = -\frac{D_{i-1}^n}{h}, \qquad s_{i,i+1} = -\frac{D_{i+1}^n}{h} \qquad \text{for } 0 < i < N.$$

At the boundaries, we will only have one integral in (14). In particular,

$$s_{0,j} = -\frac{1}{h}\int_{x_0}^{x_1} \partial_x(D^n\varphi_j) = -\frac{1}{h}\left[(D^n\varphi_j)|_{x_1} - (D^n\varphi_j)|_{x_0}\right],$$
$$s_{N,j} = \frac{1}{h}\int_{x_{N-1}}^{x_N} \partial_x(D^n\varphi_j) = \frac{1}{h}\left[(D^n\varphi_j)|_{x_N} - (D^n\varphi_j)|_{x_{N-1}}\right],$$

and so there are four boundary entries

$$s_{0,0}^n = \frac{D_0^n}{h}, \qquad s_{0,1}^n = -\frac{D_1^n}{h}, \qquad s_{N,N-1} = -\frac{D_{N-1}^n}{h}, \qquad s_{N,N} = \frac{D_N^n}{h}.$$

Now, we have all entries for the linear system. The new linear system, using $S^n = K^n + B^n$, is as follows.

1D Crank-Nicholson FEM with Piecewise Linear Finite Element Space

Given $\vec{c}^n$, solve

$$\left(M - \frac{\Delta t}{2}A^{n+1}\right)\vec{c}^{n+1} = \left(M + \frac{\Delta t}{2}A^n\right)\vec{c}^n$$

for $\vec{c}^{n+1}$ where $A^n = C^n - S^n$, and the entries of M^n, C^n, and S^n are given in Tables 1, 2, and 3.

3.1.1: Linear Test Functions and Implicit Euler Time Discretization

Following the work done for the Crank-Nicholson method, the linear system for the implicit euler time discretization with piecewise linear test functions is as follows.

1D Implicit Euler FEM with Piecewise Linear Finite Element Space

Given $\vec{c}^n$, solve the linear system

$$\left(M - \Delta t A^{n+1}\right)\vec{c}^{n+1} = M\vec{c}^n$$

for $\vec{c}^{n+1}$ where $A^n = C^n - S^n$, and the entries of M^n, C^n, and S^n are given in Tables 1, 2, and 3.

Entry of M^n	Description
$m^n_{i,i} = \begin{cases} \frac{2h}{3} & \text{for } 1 \le i \le N-1 \\ \frac{h}{3} & \text{if } i = 0, N \end{cases}$	Diagonal
$m^n_{i,i-1} = \frac{h}{6}$ for $1 \le i \le N$	Lower Diagonal
$m^n_{i,i+1} = \frac{h}{6}$ for $1 \le i \le N$	Upper Diagonal

Table 1: Entries of M^n

Entry of C^n	Description
$c_{i,i} = \begin{cases} \frac{m^n_{i-1}}{6} - \frac{m^n_{i+1}}{6} & \text{for } 1 \le i \le N-1 \\ -\frac{m^n_0}{3} - \frac{m^n_1}{6} & \text{if } i = 0 \\ \frac{m^n_{N-1}}{6} + \frac{m^n_N}{3} & \text{if } i = N \end{cases}$	Diagonal
$c_{i,i-1} = \frac{m^n_{i-1}}{3} + \frac{m^n_i}{6}$ for $0 < i \le N$	Lower Diagonal
$c_{i,i+1} = -\frac{m^n_i}{6} - \frac{m^n_{i+1}}{3}$ for $0 \le i < N$	Upper Diagonal,

Table 2: Entries of C^n

This is exactly as expected, given that the Crank-Nicholson method uses an average of A^{n+1} and A^n. We will compare the numerical accuracy of both the implicit Euler and Crank-Nicholson methods in the proceeding section. Note that D^n_i is calculated by

$$D^n_i = D(x,t)\Big|_{(x_i,t_n)} = \frac{1}{2}\boldsymbol{\sigma}\boldsymbol{\sigma}^T\Big|_{(x_i,t_n)} = \frac{1}{2}\sum_{k=1}^{m}\boldsymbol{\sigma}^2_k(x_i,t_n),$$

where $\sigma_k(x,t)$ are the noise terms that appear in the stochastic differential equation (there may be multiple sources of noise).

3.2: Numerical Simulations of the 1D Implicit Euler FEM

We consider two 1D stochastic differential equations to test the 1D finite element method, the first being the **Ornstein-Uhlenbeck (OU) process**

$$dx(t) = -ax\,dt + b\,dw_t, \quad \text{where } a \in \mathbb{R}_{\ge 0} \text{ and } b \in \mathbb{R}, \tag{15}$$

and the second being the **Cox–Ingersoll–Ross (CIR) process**

$$dx(t) = \kappa(\theta - x)\,dt + \sigma\sqrt{x}\,dw_t \quad \text{where } \theta \in \mathbb{R} \text{ and } \kappa, \sigma \in \mathbb{R}_{\ge 0}. \tag{16}$$

The Fokker-Planck equation for both the OU and CIR processes have analytical solutions, so we can compare the numerical methods with the analytical solution as a benchmark. The OU process is diffusion-dominated if $a \sim b \sim L$ (the domain length), so the CN method should outperform the implicit Euler method. On the other hand, the CIR process is convection-dominated for $x \ll 1$, so the CN method may have some oscillatory behavior near zero.

Entry of S^n	Description
$s_{ii} = \begin{cases} \frac{2D_i^n}{h} & \text{for } 1 \le i \le N-1, \\ \frac{D_i^n}{h} & \text{if } i = 0, N, \end{cases}$	Diagonal
$s_{i,i-1} = -\frac{D_{i-1}^n}{h}$ for $1 \le i \le N$	Lower Diagonal
$s_{i,i+1} = -\frac{D_{i+1}^n}{h}$ for $0 \le i \le N-1$	Upper Diagonal

Table 3: Entries of S^n

3.2.1: Ornstein-Uhlenbeck Process Simulations

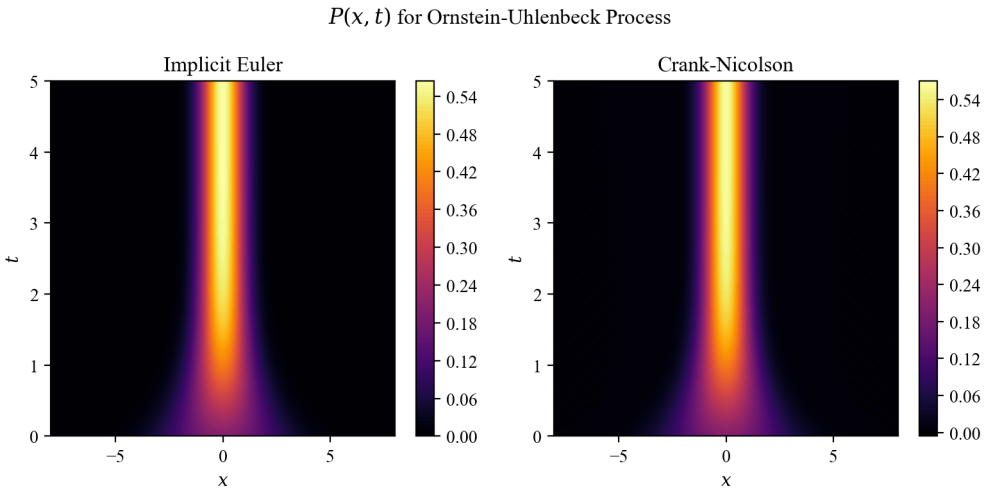


Figure 2: Numerical solutions $P(x,t)$ of the Fokker-Planck equation for the Ornstein-Uhlenbeck process using implicit Euler and Crank-Nicholson time discretizations with $\Delta t = 0.07$ and $h = 0.01$.

Simulations of both the implicit Euler and CN method for the Ornstein-Uhlenbeck process with $a = b = 1$, $\Delta t = 0.07$, and $h = 0.01$ on the domain $\Omega = [-8, 8]$ are displayed in Figure 2. There are no visible differences between the methods at the scale of Figure 2. Both converge to the stationary distribution $p_\infty(x)$ as time increases. Importantly, the CFL condition is not met in these simulations, yet the numerical methods are stable and consistent.

A comparison between the CN method, the implicit Euler method, and the true solution at $t = 5$ is displayed in Figure 5. There is no significant difference between the methods at $t = 5$, and both have very good agreement with the stationary distribution $p_\infty(x)$. The L^2 error of the implicit Euler method is 2.32×10^{-3}, and the L^2 error of the CN method is 1.08×10^{-3}. However, at $t = 10$ and all future times, both methods have an L^2 error of 0.000037. This means that both methods are accurate enough that numerical accuracy is limited by the mesh itself.

3.2.2: Cox-Ingersoll-Ross Process Simulations

Simulations of both the implicit Euler and CN method for the Cox-Ingersoll-Ross (CIR) process with

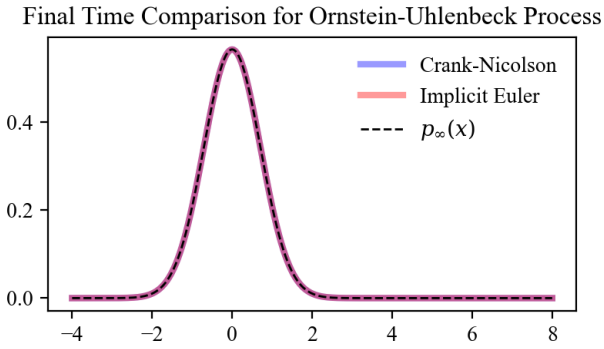


Figure 3: Comparison between the implicit Euler solution, Crank-Nicholson solution, and the true stationary distribution $p_\infty(x)$ for the Ornstein-Uhlenbeck process at $t = 5$.

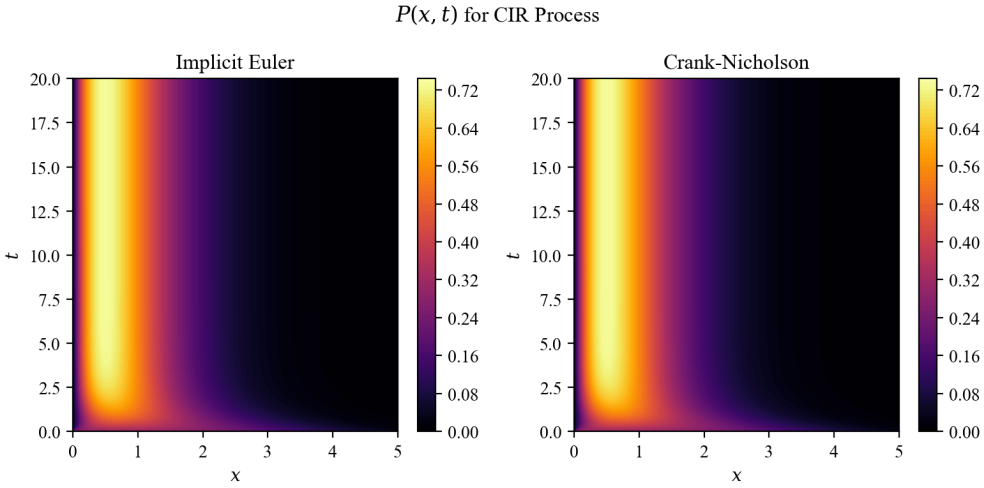


Figure 4: Numerical solutions $P(x,t)$ of the Fokker-Planck equation for the Cox-Ingersoll-Ross process using implicit Euler and Crank-Nicholson time discretizations with $\Delta t = 0.07$ and $h = 0.01$.

$\kappa = \theta = \sigma = 1$, $\Delta t = 0.07$, and $h = 0.01$ on the domain $\Omega = [0,5]$ are displayed in Figure 4. Once again, there are no visible differences between the methods at the scale of Figure 4, and both converge to the stationary distribution $p_\infty(x)$ as time increases. And, once again, the CFL condition is not met in these simulations, yet the numerical methods are stable and consistent. This is especially interesting given that the Fokker-Planck equation for the CIR process is convection-dominated near $x = 0$.

Importantly, the CN method does overshoot near $x = 0$. Across all spatial and time points , there were 23 instances of P_i^n being negative with the minimum value being -0.1607 for the CN method, while the implicit Euler method was positive for all time (and we expect a probability distribution to be positive for all x). Furthermore, the implicit Euler method has a smaller L^2 error of 6.3×10^{-4} versus 6.5×10^{-4} for the CN method at $t = 20$, likely due to the overshoot of the CN method near $x = 0$. Overshooting can be prevented by taking Δt smaller; with $h = 0.01$ fixed, overshooting occurs in the CN method for $\Delta t \geq 0.02$

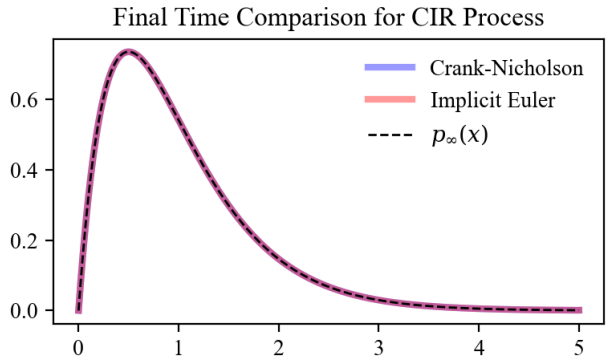


Figure 5: Comparison between the implicit Euler solution, Crank-Nicholson solution, and the true stationary distribution $p_\infty(x)$ for the CIR process at $t = 20$.

but not for $\Delta t \leq 0.01$. In general, it is not necessary to require that $\Delta t \leq h$, as there is no overshooting when $h = 0.005$ and $dt = 0.007$.

4. Discussion

In this report, I have derived, analyzed the solvability of, and programmed a finite element method for the Fokker-Planck equation. In summary, given a general Fokker-Planck equation in the form of equation (2), it can be written in flux-form as

$$p_t = -\nabla \cdot \boldsymbol{J}[p] \quad \text{where} \quad \boldsymbol{J}[p] = \boldsymbol{\mu} p - \boldsymbol{D}\nabla p - (\nabla \cdot \boldsymbol{D})p.$$

Each of the three terms in the flux operator contributes a matrix to the resulting linear system. The variational (semi-discrete) form of the finite element method is to find $P_t \in \mathbf{V}_h$ such that

$$(P_t, \varphi) = (\boldsymbol{J}[P], \nabla\varphi) \quad \text{for all } \varphi \in \mathbf{V}_h.$$

To derive the linear system, one must choose a time discretization, write P^n as a linear combination of the nodal basis functions, and extract the coefficients. I showed that under a Crank-Nicholson time discretization, the linear system is

$$\left(M - \frac{\Delta t}{2}A^{n+1}\right)\vec{c}^{\,n+1} = \left(M + \frac{\Delta t}{2}A^{n}\right)\vec{c}^{\,n} \quad \text{where} \quad A^n = C^n - K^n - B^n,$$

and the entries of M, C^n, K^n, and B^n are given in (11). I describe how to store the mesh data and compute the entries of M, C^n, K^n, and B^n for the general Fokker-Planck equation with optimal time complexity $O(N_\beta)$, where N_β is the number of nodal basis functions, although I do not go into full details on how the mesh is generated or how the integrals are computed (note that the degree of the polynomials in $\mathbf{V}_h$, r, also affects the time complexity because N_β depends on r).

For a 1D Fokker-Planck equation with uniform mesh and piecewise linear finite element space, I showed that $A^n = C^n - S^n$, and explicitly computed the entries of M, C^n, and S^n in tables 1, 2, and 3. Numerical simulations confirmed that this method is accurate and stable, even when the CFL condition

(which was shown to be sufficient for solvability) is not met. Numerical simulations also showed that the Crank-Nicholson scheme is (slightly) more accurate than implicit Euler in diffusion-dominated regions of the domain, but it can suffer from overshoot in convection-dominated regions of the domain.